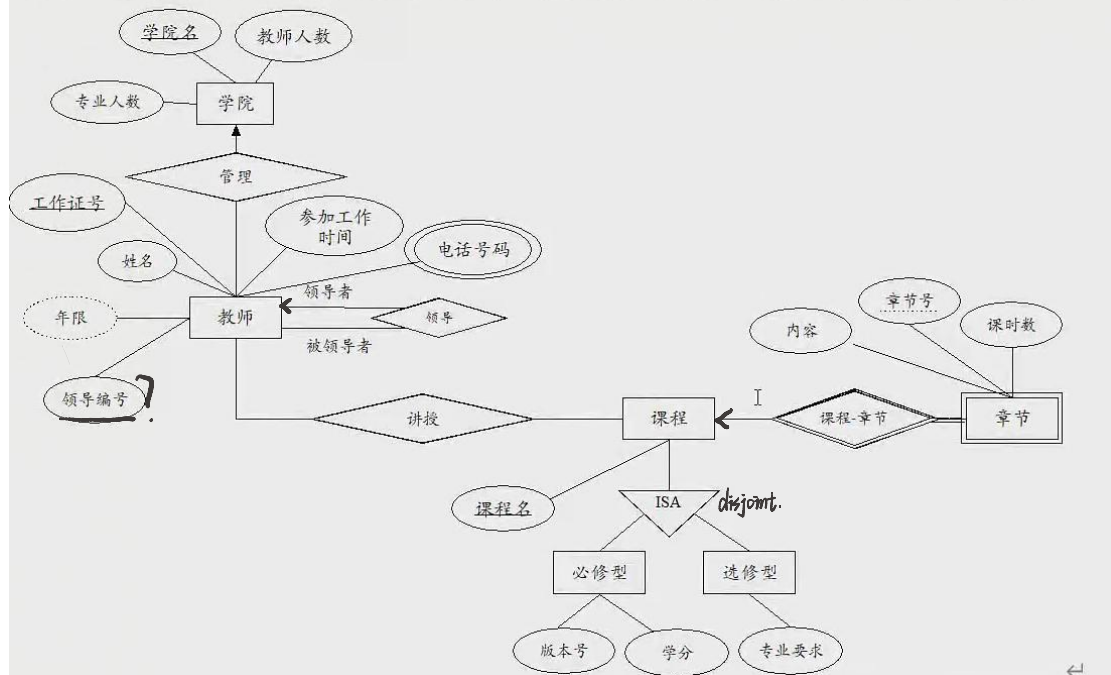


一、[14 分]根据自然语言描述绘制某高校的 E-R 图：↵

- 1、高校由多个学院(College)组成，每个学院(College)由唯一的名字(C_name)标识，还需记录专业数(Major num)、学生总人数(Stu num)及教师总人数(Tea num)等信息。↵
- 2、各学院的教师(Teacher)由其工作证号(Tea_id)来标识，还需要记录每名教师的姓名(Tea_name)、电话号码(Tea_phone)（包括所有的电话号码）、参加工作的日期(Start date)，由此日期可以推知教师的工龄(Work duration)。学院领导在教师中产生，负责领导教师日常工作。每位教师只能隶属于一个学院，且仅有一位直属领导。↵
- 3、不同课程(Course)的名称(Cou_name)也不相同。课程分为必修型(Necessary)和选修型(Selected)两种。必修型课程应记录学分(Gcredit)和课时(Cou Time)，选修型课程应记录考核方式(Assessment)。每名教师可以讲授多门课，每门课可以由多位教师讲授。↵
- 4、每门课程由若干个章节(Chapter)构成。每个章节包括章节号(Chap_id)、重要程度(Importance)和课时数(Chap_time)。虽然不同课程的章节号可能重复，但同一课程的章节号是不重复的。↵

4、每门课程由若干个章节构成。每个章节用章节号标识。需要记录每个章节的具体内容和课时数。虽然不同课程的章节号可能重复，但同一课程的章节号是不重复的。↵



- 1.主键、属性、一对多关系、角色名称、disjoint、完全参与，错 1 个，扣 1 分
- 2.角色关系、弱实体集、属性继承，错 1 个扣 3 分 ↵
- 3.不重复扣分↵

二、[12 分] 设有属于 1NF 的关系模式 $R = (A, B, C, D, E)$ ，为解题方便，假设 R 上的函数依赖集闭包 $F^+ = \{ D \rightarrow BE, A \rightarrow C, CA \rightarrow DE, D \rightarrow A \}$ ，回答以下问题。↵

1、R 是否满足 BCNF，分析原因。如果不满足请根据 F 中给出的函数依赖次序进行 BCNF 分解。[6 分]↵

回答满足 3 分，满足分析原因 3 分；判断错误，但根据分析原因情况考虑，最多 3 分。

2、R 是否满足 3NF，分析原因。如果不满足请根据 3NF 分解算法对关系模式 R 进行分解。[6 分]↵

回答满足 3 分，分析原因 3 分，；判断错误，但分析原因情况考虑，最多 3 分。↵

三、[18 分，每小题 3 分] 根据要求写出关系代数表达式：↵

- $branch = (branch_name, branch_city, assets)$ ↵
- $customer = (customer_id, customer_name, customer_street, customer_city)$ ↵
- $loan = (loan_number, amount, branch_name)$ ↵
- $account = (account_number, balance, branch_name)$ ↵
- $borrower = (customer_id, loan_number)$ ↵
- $depositor = (customer_id, account_number)$ ↵

1、查询所有在 London 支行没有贷款帐户的客户编号↵

$\Pi_{customer_id}(customer) - \Pi_{customer_id} \sigma_{branch_name = "London"}(branch \bowtie loan \bowtie borrower)$

2、查询在 Brooklyn 城市的每个支行都有存款帐户的客户编号↵

$\Pi_{customer_id, branch_name}(account \bowtie depositor) \div \Pi_{branch_name} \sigma_{branch_city = "Brooklyn"}(branch)$ ↵

3、查询 Brooklyn 城市各支行的存款帐户额最大值。↵

$branch_name \ g \ max(balance) (\sigma_{branch_city = "Brooklyn"}(branch \bowtie account))$ ↵

4、将存款帐号为 A001 的所有相关存款信息从数据库中删除↵

$account \leftarrow account - \sigma_{account_number = "A001"}(account)$ ↵

$depositor \leftarrow depositor - \sigma_{account_number = "A001"}(depositor)$ ↵

5、给 Perryridge 支行所有贷款客户提供额度为 200 的存款账户。该存款账户的账号为相应贷款客户的贷款帐号。↵

$r_1 \leftarrow (\sigma_{branch_name = "Perryridge"}(borrower \bowtie loan))$ ↵

$account \leftarrow account \cup \Pi_{loan_number, branch_name, 200}(r_1)$ ↵

$depositor \leftarrow depositor \cup \Pi_{customer_id, loan_number}(r_1)$ ↵

6、删除 Perryridge 支行的所有存款记录。↵

$account \leftarrow account - \sigma_{branch_name = "Perryridge"}(account)$ ↵

↵

四、[24 分] 关系表结构与上题相同，试用 SQL 语句表达（其中查询语句只能用一条语句实现，全部正确每小题 3 分。表达与标准答案不一致，但是正确仍给 3 分）：↵

1、查询存款账号数量多于 1000（不包含 1000），并且名称以 'CH' 结尾的支行名称↵

Select branch_name, count(account_number)↵

From account↵

Where branch_name like '%CH'↵

```
Group by branch_name ←  
Having count(account_number)>1000←
```

- 2、编号为 001 的客户在 Brooklyn 城市多个支行有贷款账户，查询在所有这些支行中也均有贷款帐户的客户编号←

```
Select A.customer_id ←  
From customer A←  
Where not exists ( select branch_name←  
From loan, borrower←  
Where borrower.loan_number=loan.loan_number ←  
And customer_id='001' and branch_name=' Brooklyn')←  
Except←  
( select branch_name←  
From loan, borrower B←  
Where B.loan_number=loan.loan_number ←  
And B.customer_id= A.customer_id)←
```

- 3、在数据库中加入如下信息：编号为 A-973 的客户在 Perryridge 支行建立贷款帐户，帐户编号为 0019，贷款额为 1200←

```
Insert into loan(loan_number, branch_name, amount) values ('0019', 'Perryridge',  
1200)←  
Insert into borrower(loan_number, customer_id) values ('0019', 'A-973')←
```

- 4、把 Perridge 支行所有存款账户额提高 10%←

```
Update account ←  
Set balance=balance*1.1←  
Where branch_name=' Perridge'←
```

- 5、删除 branch 表中的 branch_city 属性←

```
Alter table branch drop branch_city←
```

- 6、查询比 Brooklyn 城市所有支行的资产都多的支行名称。←

```
select branch_name↓  
from branch↓  
where assets > all↓  
(select assets↓  
from branch↓  
where branch_city = 'Brooklyn') ←
```


- 7、用 DDL 语言实现 *branch* 表，属性类型自定义。↵ I
- Create table branch↵
 (branch_name char(20) primary key,↵
 Branch_city char(20)↵
 Assets integer)↵
- 8、将在 *branch* 表的 select、update、delete 权利授予给 James↵
 Grant select, update, delete on branch to James↵

五、[14 分，每小题 7 分] 1、绘制优先图并分析下图所示各事务是否满足冲突可串行

不满足冲突可串行化 3 分，分析原因 4 分↵
 2、说明视图可串行化调度应满足的条件。请给出下图所示各事务的视图可串行化调度次序。要求给出所有可能的次序。↵

T_1 ↵	T_2 ↵	T_3 ↵	T_4 ↵	T_5 ↵	↵
Read(Y)↵	↵	↵	↵	↵	↵
↵	Write(Y)↵	↵	↵	↵	↵
↵	↵	Read(Y)↵	↵	↵	↵
Write(Y)↵	↵	↵	↵	↵	↵
↵	↵	↵	Read(Z)↵	↵	↵
Read(X)↵	↵	↵	↵	↵	↵
↵	↵	Write(Y)↵	↵	↵	↵
↵	↵	↵	Read(X)↵	↵	↵
↵	↵	Write(Z)↵	↵	↵	↵
↵	↵	↵	Write(X)↵	↵	↵
↵	↵	↵	↵	Read(Z)↵	↵
↵	↵	↵	↵	Write(Z)↵	↵

满足冲突可串行化 3 分， 给出串行化次序 4 分↵

T1 T2 T4 T3 T5 T1 T4 T2 T3 T5↵
 判断错误但分析原因，根据情况最高 3 分 ↵

- 六、[18 分，每小题 6 分] 论述说明题：↵ I
- 1、说明数据库三级抽象。↵
 物理层、逻辑层、视图层 每方面介绍 3 分↵
- 2、举例说明事务的 ACID 特征。↵
 原子性、一致性、隔离性、可持久性 每方面介绍 1.5 分↵
- 3、在数据库 E-R 图设计过程中，如何区别使用实体集和属性、实体集和联系集、强实体集和弱实体集？请举例说明↵
 每个方面 2 分，要求举例适当、详尽。如果不举例，仅概念性介绍最多给 3 分。